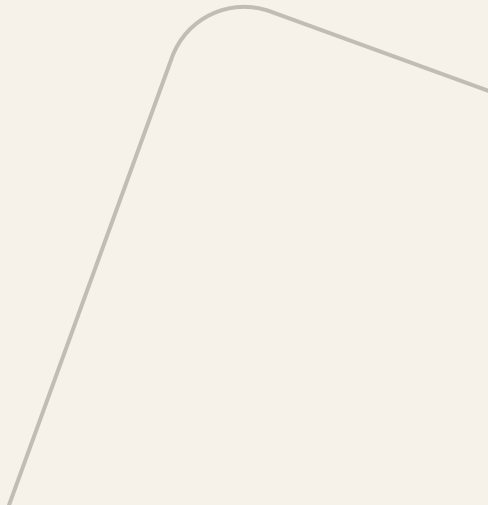


SWE 4101

Object Oriented Concepts I

Prepared By:
Samia Nawsheen
Junior Lecturer, CSE



Class 1

Objectives

01 The problem: duplicated classes

02 The fix: a generic class

03 Using generics + the diamond operator

04 Generics with collections

05 Why not just use Object?

06 Bounded generics

The Problem: One Class Per Type

- Say we want a class that just holds one value and prints it out
- We build IntegerPrinter
- This class does exactly one thing: hold a value, print a value

```
public class IntegerPrinter {  
    Integer thingToPrint;  
  
    public IntegerPrinter(Integer thingToPrint) {  
        this.thingToPrint = thingToPrint;  
    }  
  
    public void print() {  
        System.out.println(thingToPrint);  
    }  
}
```

Before Generics: Copy, Paste, Repeat

- What if we need a Double Printer or a String Printer?
- To do the same for Double, we'd copy this whole class and rename every Integer to Double
- Doing that again for String makes three nearly identical classes

Question for the class

IntegerPrinter, DoublePrinter, and StringPrinter would all have the exact same fields, constructor, and method — just with one word changed throughout. What's actually different between these three classes?

Working it through

- Nothing about the logic changes at all
- Only "Integer" vs "Double" vs "String"
- We're rewriting identical code three times, just to swap a single type
- We want to write the logic once, and plug in the type later

The Fix: A Generic Class

```
public class Printer<T> {  
    T thingToPrint;  
  
    public Printer(T thingToPrint) {  
        this.thingToPrint = thingToPrint;  
    }  
  
    public void print() {  
        System.out.println(thingToPrint);  
    }  
}
```

- <T> declares a placeholder type, called a **type parameter**
- T is just a name, by convention, short for "Type", not a special keyword

The Fix: A Generic Class

- Same exact logic as IntegerPrinter
- But T replaces every Integer
- This one class can now do the job of IntegerPrinter, DoublePrinter, and StringPrinter combined
- We can delete all three old classes entirely

Using Printer<T>

```
Printer<Integer> intPrinter = new Printer<>(23);  
intPrinter.print();  
  
Printer<Double> doublePrinter = new Printer<>(33.5);  
doublePrinter.print();  
  
Printer<String> stringPrinter = new Printer<>("Hello, Generics!");  
stringPrinter.print();
```

- Printer<Integer> fills in T as Integer, just for this one variable
- new Printer<>() — the empty <> is the diamond operator; Java infers the type from the left side
- Older Java made you repeat the type on both sides; the diamond operator got rid of that

Common mistake

- Generics don't work with primitive types like **int** or **long**
- Use the wrapper class instead: Integer instead of int, Long instead of long
- `Printer<int>` won't compile
- `Printer<Integer>` will

Generics in Collections

- You've already used generics without naming them in ArrayLists
- Suppose, we have an ArrayList of Cat objects
- ArrayList<Cat> means "this list is only allowed to hold Cat objects"
- Adding anything else, like a Dog will give a compile error

```
ArrayList<Cat> cats = new ArrayList<>();  
cats.add(new Cat());    // fine  
cats.add(new Dog());    // compile error!
```

Question for the class

Why not just make every list hold plain Object, and let it hold all kinds of objects (String, Integer, Cat, Dog, etc.)?

Without Generics: The Object Trick

```
ArrayList<Object> rawList = new ArrayList<>();    // no type at all
rawList.add(new Cat());

Cat myCat = (Cat) rawList.get(0);    // must cast
```

- rawList only knows it holds Objects
- We can add any object to it
- Getting myCat back out requires a cast
- So far this looks fine

Working it through

- We can't be sure of what's actually inside the list
- Getting anything back out would require a cast
- Java can't verify that the cast will make sense
- Let's see exactly what goes wrong when we try this

The Crash Waiting to Happen

```
rawList.add(new Dog());    // compiles fine -- rawList holds anything
```

```
Cat myCat = (Cat) rawList.get(1);    // ClassCastException at runtime!
```

- Nothing stops a Dog from being added to this list
- The cast to Cat compiles perfectly
- Java has no way to check it in advance
- The program only crashes once it actually runs and hits this line

Generics Fix This

- `ArrayList<Cat>` makes the compiler enforce the promise, instead of trusting you to keep it
- Adding a `Dog` is now caught immediately, while writing the code
- Getting a `Cat` back out never needs a cast
- Java already knows it can only be a `Cat`

Bounded Generics: extends

- Sometimes you want a generic type restricted to only certain kinds of types
- **◁T extends Animal▷** means "T can be anything, as long as it's an Animal or a subclass of Animal"
- **◁T extends Flyable▷** means "T can be anything, as long as it's Flyable or a class that implements the Flyable interface"
- Now the class can rely on Animal or Flyable's own methods being available on T

A Printer Just for Animals

```
public class Printer<T extends Animal> {  
    T thingToPrint;  
  
    public void print() {  
        thingToPrint.eat();    // safe -- every T is an Animal  
    }  
}  
  
Printer<Cat> catPrinter = new Printer<>(new Cat());  
Printer<Integer> intPrinter = new Printer<>(23);    // compile error now!
```

- Because every possible T is guaranteed to be an Animal, calling eat() is safe
- Cat and Dog both extend Animal, so they are allowed; but not Integer

Question for the class

extends is normally for classes only, implements is for interfaces. But you can also write <T extends Flyable>, even though Flyable is an interface. Why extends here, and not implements?

Working it through

- In a generic bound, extends is used for both classes and interfaces
- Java has no separate keyword for this spot
- It just means "T must be, or must implement this type"
- It looks a little unusual, but implements is never valid here, even for an interface

Multiple Bounds

```
public class Printer<T extends Animal & Flyable> {  
    T thingToPrint;  
    // ...  
}
```

- & lets you specify more than one bound at once
- T must satisfy all of them
- You can only list one class this way, since Java doesn't allow multiple inheritance
- You can list multiple interfaces
- Any interfaces must come after the class, always separated by &

Common mistake

// Wrong order:

```
class Printer<T extends Flyable & Animal> { }
```

// Correct order -- class first, then interfaces:

```
class Printer<T extends Animal & Flyable> { }
```

- The class must always come first in a multiple bound
- Interfaces come after
- Flyable & Animal fails to compile purely because of the order

Thank You

Class 2

Objectives

- 01** Recap: generic classes & bounds
- 02** Generic methods
- 03** Multiple type parameters

- 04** Returning a generic type
- 05** Wildcards: ?
- 06** Bounded wildcards

Recap

- A generic class uses a placeholder type, like `Printer<T>`
- extends restricts what T is allowed to be, so the class can safely call a method that class is guaranteed to have
- Multiple bounds (`T extends X & Y`) means T must satisfy all of those types at once

Generics Work in Interfaces Too

- An interface can declare a placeholder type, exactly like a class
- Every method inside the interface can use that T
- A class can implement it and either fix a concrete type, or stay generic itself

A Generic Interface, Implemented

```
interface Container<T> {  
    T get();  
    void set(T value);  
}  
  
class Box<T> implements Container<T> {  
    private T value;  
    public T get() { return value; }  
    public void set(T value) { this.value = value; }  
}  
  
class StringContainer implements Container<String> {  
    private String value;  
    public String get() { return value; }  
    public void set(String value) { this.value = value; }  
}  
  
// Box stays generic  
// StringContainer becomes concrete
```

The Problem: A Method for Any Type

- Say we want a method that prints anything, followed by exclamation points
- We don't want a separate shout(String), shout(Integer), shout(Cat)...
- We want one method that works for any type we hand it

A Generic Method

```
private <T> void shout(T thingToShout) {  
    System.out.println(thingToShout + "!!!");  
}  
  
shout("John");  
shout(57);  
shout(new Cat());
```

- <T> right before the return type declares this method's own placeholder
- This T belongs only to this method, separate from any class-level T
- Java figures out T automatically from whatever you pass in

A Generic Method

- `Printer<T>`'s own `print()` method was not a generic method
- It just reused the `T` that `Printer<T>` had already declared
- A method only needs its own `<T>` if it isn't already inside a generic class, or needs a different placeholder

A Static Method Can't Borrow the Class's T

```
class Printer<T> {  
    T thingToPrint;  
  
    static void bad(T item) { ... }           // compile error!  
    static <T> void good(T item) { ... }      // fine -- its own T  
}
```

- T only means something once a specific Printer<String> or Printer<Integer> is created
- A static method can be called as Printer.good(...) without ever creating a Printer at all
- So a static method can't see the class's T — it must declare its own, even naming it T again

Type Erasure

```
class Test<T> {  
    static int count = 0;  
    Test() { count++; }  
}  
  
Test<Integer> obj1 = new Test<>();  
Test<String> obj2 = new Test<>();  
Test<Double> obj3 = new Test<>();  
System.out.println(Test.count);    // 3
```

- Java forgets the generic type at runtime – Type Erasure
- Test<Integer>, Test<String>, Test<Double> are really just one class in memory
- count is static and belongs to the one shared class, not any particular T

Adding an Extra Placeholder of Your Own

```
class Printer<T> {  
    T thingToPrint;  
  
    <F> void printWithLabel(F label) {  
        System.out.println(label + ": " + thingToPrint);  
    }  
}
```

- printWithLabel uses both F (its own) and T (the class's) at the same time
- F is scoped only to this one method — the class itself stays Printer<T>, never Printer<T, F>
- A method can always add its own placeholder on top, for something the class-level one doesn't cover

Multiple Type Parameters in a Method

```
private static <T, V> void shout(T thingToShout, V otherThingToShout) {  
    System.out.println(thingToShout + "!!!");  
    System.out.println(otherThingToShout + "!!!");  
}  
  
shout("John", 74);
```

- A method can declare more than one placeholder, separated by commas
- T and V are tracked completely independently
- The same idea works for generic classes too
- Printer<T, V> class with two placeholders

Returning a Generic Type

```
private static <T> T echo(T thingToShout) {  
    return thingToShout;  
}
```

```
String s = echo("hello");  
int n = echo(57);
```

- The return type can be T itself
- Whatever type goes in is exactly the type that comes back out
- No casting needed
- Java already knows the return type matches the argument's type

Returning One of Multiple Generic Types

```
static <T, V> T pickFirst(T first, V second) {  
    return first;  
}  
  
String s = pickFirst("hello", 42);    // T=String, V=Integer
```

- T and V are tracked independently, no matter how many placeholders a method declares
- Declaring the return type as exactly T tells Java precisely what this call resolves to — no cast

The Problem: A List of... Anything

- Say we want a method that can print out a list, no matter what it holds
- We don't want `printList(List<Integer>)`, `printList(List<Cat>)`, `printList(List<String>)`...
- We want one method that accepts a list of any type at all

Common mistake

```
static void printList(List<Object> myList) { ... }  
  
List<Integer> intList = new ArrayList<>();  
printList(intList);    // compile error!
```

- It seems like this should work
- Integer is a subclass of Object, after all
- But List<Integer> is not a subclass of List<Object>
- Generics don't work that way
- We need a different tool to say "a list of some unknown type"

Wildcards: ?

- A wildcard (?) means "some type, and I don't need to know which"
- List<?> means "a list of something, any single type, just not mixed"
- Used when a method just needs to use a list generically, without caring what's inside

Using a Wildcard

```
static void printList(List<?> myList) {  
    System.out.println(myList);  
}  
  
printList(intList);    // a List<Integer> works  
printList(catList);    // a List<Cat> works too
```

- List<?> accepts a list of any single type — Integer, Cat, String, anything
- Unlike List<Object>, this genuinely works with a List<Integer> or List<Cat>
- One method now handles every possible kind of list

Couldn't We Just Use List<T>?

```
static <T> void printList(List<T> myList) {  
    System.out.println(myList);  
}  
  
static void printList(List<?> myList) {  
    System.out.println(myList);  
}
```

- Both compile, both accept a List<Integer> or a List<Cat> just fine
- Use <T> when the type needs to be reused elsewhere — as a return type, or a second parameter that must match
- Use ? when the type is never referred to again — shorter, and it says "this method doesn't care what the type is"

Bounded Wildcards

- **Upper Bounded Wildcard:** `List<? extends Animal>` means "a list of some type, but that type must be an Animal"
- **Lower Bounded Wildcard:** `? super Dog` means "some type that a Dog can safely be put into" – that is Dog itself, or any of its supertypes

Upper Bounded Wildcard

```
static void printAnimals(List<? extends Animal> myList) {  
    System.out.println(myList);  
}  
  
printAnimals(catList);    // works -- Cat extends Animal  
printAnimals(intList);    // compile error -- Integer isn't an Animal
```

- catList is allowed, because Cat satisfies the Animal bound
- intList is rejected, because Integer doesn't extend Animal
- This combines the flexibility of a wildcard with the safety of a bound

Upper vs Lower Bound

```
void feedAnimals(List<? extends Animal> animals) {  
    for (Animal a : animals) a.eat();    // safe to read  
    // animals.add(new Dog());    // compile error  
}  
  
void addDogs(List<? super Dog> animals) {  
    animals.add(new Dog());    // safe  
    animals.add(new Puppy());    // safe -- Puppy extends Dog  
    // animals.add(new Cat());    // compile error  
    // for (Dog d : animals) d.eat();    // compile error  
}
```

- feedAnimals only reads — extends guarantees every item is at least an Animal
- addDogs only adds — super guarantees Dog and its subtypes safely fit inside

Benefits of Generics



Type Safety

Wrong types are caught at compile time, not runtime



Code Reuse

One class or method works for every type



Less Boilerplate

No more copy-pasting near-identical classes



No Casting

Get values back out without manual casts

Practice exercise

- 1 Write a generic method `describe(T item)` that prints "Item: " followed by the item
- 2 Give it two type parameters instead of one, and print both
- 3 Write a method `first(T[] array)` that returns `array[0]`

Thank You